

Unit -5

UI Fragments, Menus and Dialogs

The Need for UI Flexibility

UI Flexibility is a critical aspect of modern Android development, ensuring that applications provide a seamless and consistent user experience across a wide range of devices with varying screen sizes and orientations. Given the diversity of Android devices, from smartphones and tablets to foldable screens and televisions, developers must design UIs that adapt to these differences effectively.

Challenges in Achieving UI Flexibility:

- **Designing for Different Screen Sizes:**

Android devices come in a wide array of screen sizes, from small phones to large tablets. Each screen size demands a layout that is easy to navigate and visually appealing. A UI designed for a small phone screen might look cluttered or overly simplistic on a large tablet, while a design optimized for a tablet might not fit or appear cramped on a smaller phone.

- **Managing Layouts for Different Orientations:**

Devices can be used in both portrait and landscape orientations. A layout that works well in portrait mode might not translate seamlessly to landscape mode. For instance, a single-column list in portrait might need to become a multi-column grid in landscape. Developers must anticipate these changes and design layouts that can dynamically adjust to orientation shifts without disrupting the user experience.

- **Handling Dynamic Content:**

Applications often need to display dynamic content, such as lists, images, or interactive elements that vary in size and quantity. For instance, a news app may display articles in a list, but on a larger screen, it might show a list alongside a detailed view of the selected article. Handling such dynamic content requires careful consideration of how UI components are arranged and resized across different devices.

Solution: Fragments for UI Modularity and Reusability

To address these challenges, Android provides Fragments—modular components that represent portions of the user interface within an activity. Fragments allow developers to build flexible and reusable UI components that can be combined in various ways to create layouts that adapt to different screen sizes and orientations.

- **Modularity:**

Fragments enable developers to break down complex UIs into smaller, manageable pieces. Each fragment can encapsulate a specific part of the UI, such as a list of items or a detailed view of a single item. These fragments can be reused across multiple activities or combined in different ways depending on the screen size and orientation.

- **Reusability:**

Once a fragment is created, it can be reused in different activities or even within the same activity under different configurations. For example, a fragment that displays a list of items can be used in both a single-pane layout for phones and a dual-pane layout for tablets, where it is displayed alongside another fragment showing item details.

- **Dynamic UI Adjustments:**

Fragments allow for dynamic adjustments to the UI. For example, on a phone, an app might use a single fragment to display a list of items, but on a tablet, it might use two fragments side by side—one for the list and another for detailed information about the selected item. This flexibility is crucial for maintaining a consistent user experience across devices.

- **Orientation Changes:**

Fragments also make it easier to handle orientation changes. When the device orientation changes, the activity can replace or rearrange fragments to better fit the new screen dimensions. This ensures that the UI remains usable and aesthetically pleasing in both portrait and landscape modes.

By leveraging fragments, developers can create adaptive UIs that cater to the diverse range of Android devices, ensuring that users have a consistent and high-quality experience, regardless of the device they are using. This approach not only enhances the user experience but also simplifies the development process by promoting code reuse and modular design.

Introduction to Fragments

Fragments are a fundamental component in Android development, introduced to address the need for more modular and flexible user interfaces. A fragment represents a portion of the user interface within an activity, similar to an activity itself but designed to be reusable and adaptable. Unlike activities, which typically represent a single, complete screen, fragments are smaller building blocks that can be combined, reused, and managed independently within an activity.

What is a Fragment?

A **Fragment** is a modular section of an activity that has its own lifecycle, can handle its own user input, and is capable of being combined with other fragments to create a more dynamic and flexible UI. Fragments are like "sub-activities" that can be added or removed from an activity dynamically while the activity is running. This makes them especially useful for creating multi-

pane UIs, where different fragments can be displayed side by side or in other configurations, depending on the device's screen size and orientation.

Purpose and Advantages of Using Fragments:

1. Breaking Down Complex UIs:

- **Simplification:** In many applications, the user interface can become complex, especially when dealing with various components such as lists, forms, and detail views. Fragments allow developers to break down these complex UIs into simpler, more manageable pieces. For example, a shopping app might have separate fragments for displaying product categories, product details, and the shopping cart. Each fragment handles its specific task, making the code easier to manage and maintain.
- **Isolation:** Since each fragment is self-contained, it can encapsulate all the logic and UI elements related to a specific part of the application. This isolation ensures that changes in one fragment do not unintentionally affect other parts of the UI, leading to more robust and maintainable code.

2. Displaying Multiple Fragments on a Single Screen:

- **Enhanced Multi-Panel Layouts:** One of the primary benefits of fragments is their ability to be displayed simultaneously on a single screen. This is particularly useful for larger devices like tablets, where more screen real estate is available. For instance, in an email application, one fragment could display the list of emails on the left side of the screen, while another fragment displays the content of the selected email on the right side. This side-by-side view enhances user experience by allowing users to interact with different parts of the app simultaneously.
- **Dynamic Configuration:** Fragments provide the flexibility to rearrange or replace components dynamically. Depending on the screen size or orientation, an activity can add, replace, or remove fragments to provide the best possible user experience. For example, in landscape mode, a two-pane layout might be used, while in portrait mode, the app might switch to a single-pane layout, where the user navigates between fragments.

3. Reusing Fragments Across Different Activities:

- **Code Reusability:** Fragments are designed to be reusable components. Once a fragment is created, it can be used across multiple activities, allowing developers to avoid duplicating code and ensuring consistency across the app. For example, a fragment that displays a user profile can be reused in different parts of the app, whether it's within the main dashboard, a user settings page, or a social feed.
- **Consistency:** By reusing fragments, the UI remains consistent across different parts of the application. Users experience the same interface elements and behavior, regardless of where they encounter them in the app. This leads to a more coherent and predictable user experience.

4. Enhanced Flexibility and Responsiveness:

- **Adaptability:** Fragments enable the application to adapt to different devices and configurations. Whether the app is running on a small phone or a large tablet, fragments can be used to create UIs that respond to the available screen space. Developers can design their apps to show more information on larger screens while keeping the interface clean and focused on smaller screens.

- **Smooth Transitions:** Since fragments can be added, removed, or replaced dynamically, they allow for smooth transitions between different UI states without requiring the activity itself to be restarted. This makes the app feel more responsive and fluid, improving the overall user experience.

Lifecycle of a Fragment

Understanding the **lifecycle of a fragment** is crucial for managing its state effectively, handling user interactions, and ensuring optimal resource management. The fragment lifecycle consists of a series of callback methods that the system invokes at different stages of a fragment's existence. Each lifecycle method serves a specific purpose and provides an opportunity to manage various aspects of the fragment. Below is a detailed description of each lifecycle method:

1. **onAttach()**

- **Purpose:** This method is called when the fragment is first attached to its host activity. It occurs before the fragment's view is created.
- **Usage:** Use this method to perform operations that need access to the activity context, such as initializing data or interacting with the activity. It's the first callback where the fragment can access its parent activity.
- **Example:** You might use `onAttach()` to set up communication between the fragment and the activity, or to initialize context-dependent resources.

2. **onCreate()**

- **Purpose:** This method is called to initialize the fragment. It's where you should perform one-time initialization, such as setting up essential components and restoring the previous state.
- **Usage:** Here, you can initialize fragment-specific variables, set up configurations, or restore saved instance states if needed.
- **Example:** You could initialize a `ViewModel` or configure a database connection in this method.

3. **onCreateView()**

- **Purpose:** Called to create and return the view hierarchy associated with the fragment. This method is crucial for inflating the fragment's layout.
- **Usage:** Inflate the layout XML file for the fragment and return the root view. This is where you define the fragment's UI components.
- **Example:** You would typically use `LayoutInflater` to inflate the fragment's layout:
`return inflater.inflate(R.layout.fragment_layout, container, false);`

4. **onActivityCreated()**

- **Purpose:** Invoked when the fragment's activity has been created and its view hierarchy has been instantiated. It occurs after `onCreateView()`.
- **Usage:** Use this method to perform final initialization related to the activity's view hierarchy. It's a good place to set up interactions between the fragment's UI and the activity.
- **Example:** Initialize UI components or interact with the activity's views, such as setting up adapters or data bindings.

5. **onStart()**

- **Purpose:** Called when the fragment becomes visible to the user. It marks the point when the fragment starts to interact with the user.
- **Usage:** Perform actions that need to occur each time the fragment becomes visible. This is where you start operations that need to be resumed when the fragment is visible.
- **Example:** Start animations or register listeners that should be active while the fragment is visible.

6. **onResume()**

- **Purpose:** Called when the fragment is visible and interacting with the user. It indicates that the fragment is in the foreground and actively interacting with the user.
- **Usage:** Resume any UI updates, animations, or functionality that was paused or stopped during `onPause()`.
- **Example:** Resume a video playback, restart animations, or reattach listeners that were detached in `onPause()`.

7. **onPause()**

- **Purpose:** Called when the fragment is no longer interacting with the user. It happens when another activity or fragment comes into the foreground.
- **Usage:** Save data, pause animations, or release resources that are not needed while the fragment is not in the foreground.
- **Example:** Pause ongoing tasks like animations or background operations, save user inputs, or stop sensor updates.

8. **onStop()**

- **Purpose:** Called when the fragment is no longer visible to the user. It occurs when the fragment is no longer in the foreground.
- **Usage:** Release resources that are not needed when the fragment is not visible. Perform cleanup tasks that should occur when the fragment is not in use.
- **Example:** Unregister listeners, release large resources, or stop background services that are only needed while the fragment is visible.

9. **onDestroyView()**

- **Purpose:** Called when the view hierarchy associated with the fragment is being removed. This happens before the fragment is destroyed.
- **Usage:** Perform cleanup related to the fragment's view, such as removing view references or stopping view-related tasks.
- **Example:** Release resources related to the view, such as canceling animations or removing callbacks that are tied to the view.

10. **onDestroy()**

- **Purpose:** Called when the fragment is being destroyed. This method is invoked when the fragment is no longer needed and is about to be removed.
- **Usage:** Perform final cleanup of any remaining resources that were allocated during the fragment's lifecycle.
- **Example:** Clean up or release resources like database connections, network operations, or any other resources that were allocated during the fragment's lifecycle.

11. **onDetach()**

- **Purpose:** Called when the fragment is detached from its host activity. This is the final callback before the fragment is completely removed.
- **Usage:** Perform any final cleanup that requires the context of the activity. It's the last opportunity to interact with the activity before the fragment is fully detached.
- **Example:** Release references to the activity or perform any necessary cleanup related to the activity context.

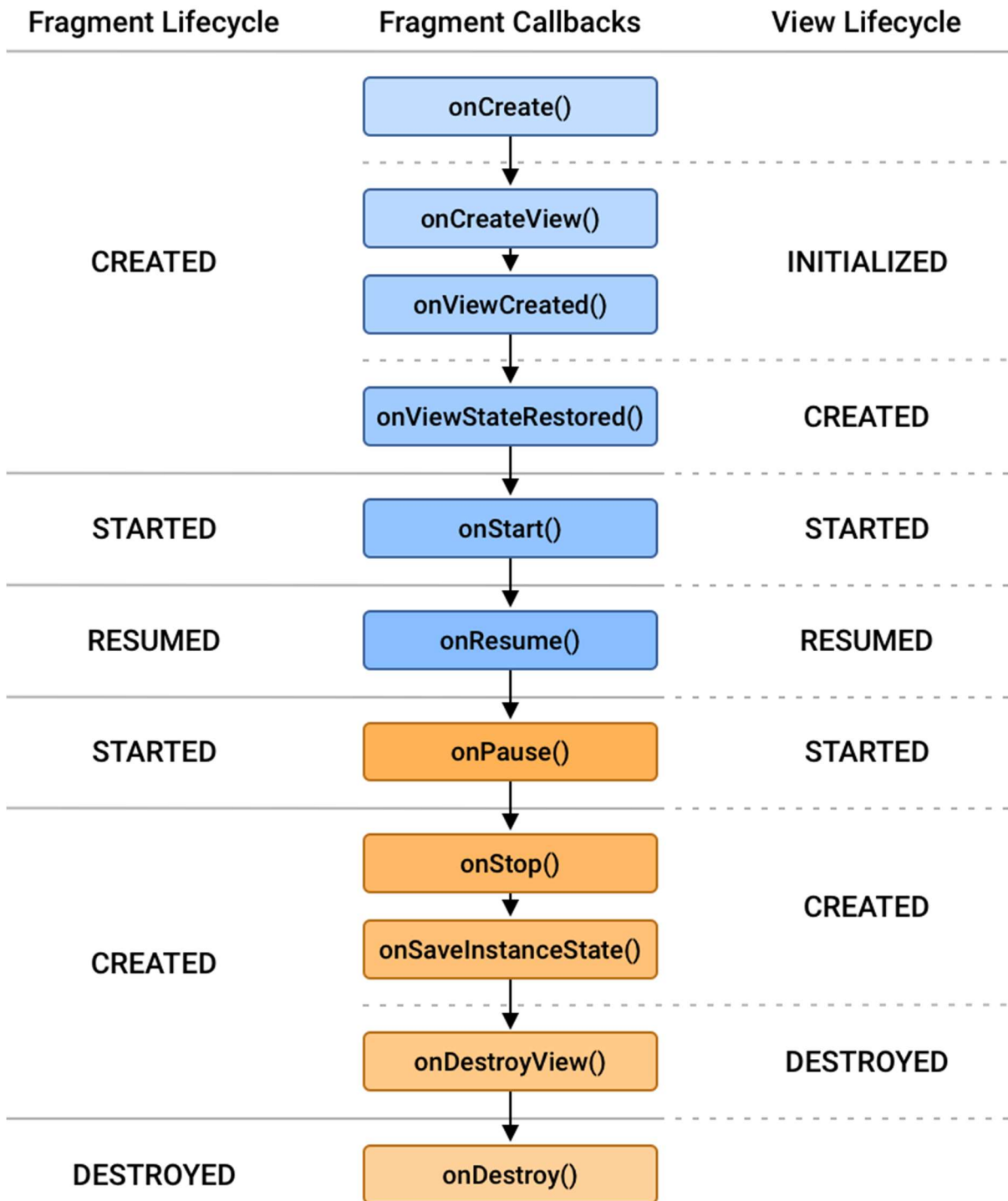


Figure 1. *Fragment Lifecycle states and their relation to both the fragment's lifecycle callbacks and the fragment's view Lifecycle.*

Creating a UI Fragment

1. Creating a Fragment Class:

- Extend `Fragment` class.
- Override lifecycle methods such as `onCreateView()` to define the UI.

```
public class MyFragment extends Fragment {
    @Nullable
    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup
container, Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_layout, container,
false);
    }
}
```

2. Wiring Widgets in a Fragment:

- Use `findViewById()` within `onCreateView()` to interact with UI components in the fragment.

```
@Override
public View onCreateView(LayoutInflater inflater, ViewGroup container,
Bundle savedInstanceState) {
    View view = inflater.inflate(R.layout.fragment_layout, container,
false);
    TextView textView = view.findViewById(R.id.textView);
    textView.setText("Hello, Fragment!");
    return view;
}
```

Building a Multi-Panel User Interface for a Tablet App

In this scenario, we are developing a tablet application for a recipe book. The application displays a list of recipes and details of the selected recipe. On tablets, we want to show both the list of recipes and the details side-by-side, while on phones, we want to display them in a single-pane layout, switching between the list and detail views.

We will use fragments to achieve this UI flexibility. The main activity will handle the fragment transactions to display the appropriate layout based on the screen size.

1. Define Fragments

1.1. RecipeListFragment: Displays a list of recipes.

1.2. RecipeDetailFragment: Displays details of the selected recipe.

2. Create the Fragments

RecipeListFragment.java

```
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.ArrayAdapter;
import android.widget.ListView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

public class RecipeListFragment extends Fragment {

    private OnRecipeSelectedListener callback;

    @Override
    public void onAttach(@NonNull Context context) {
        super.onAttach(context);
        // Ensure that the host activity implements the callback interface
        try {
            callback = (OnRecipeSelectedListener) context;
        } catch (ClassCastException e) {
            throw new ClassCastException(context.toString() + " must implement OnRecipeSelectedListener");
        }
    }

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_recipe_list, container, false);

        // Initialize ListView and its adapter
        ListView listView = view.findViewById(R.id.recipe_list);
        String[] recipes = {"Spaghetti Bolognese", "Chicken Alfredo", "Beef Stroganoff"};
        ArrayAdapter<String> adapter = new ArrayAdapter<>(getActivity(),
            android.R.layout.simple_list_item_1, recipes);
        listView.setAdapter(adapter);

        // Set an item click listener
        listView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
            @Override
```



```

        public void onItemClick(AdapterView<?> parent, View view, int
position, long id) {
            // Notify the host activity about the selected recipe
            callback.onRecipeSelected(position);
        }
    });

    return view;
}

// Interface to communicate with the host activity
public interface OnRecipeSelectedListener {
    void onRecipeSelected(int position);
}
}

```

RecipeDetailFragment.java

```

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

public class RecipeDetailFragment extends Fragment {

    private static final String ARG_POSITION = "position";

    public static RecipeDetailFragment newInstance(int position) {
        RecipeDetailFragment fragment = new RecipeDetailFragment();
        Bundle args = new Bundle();
        args.putInt(ARG_POSITION, position);
        fragment.setArguments(args);
        return fragment;
    }

    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable
ViewGroup container,
                            @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_recipe_detail, container, false);

        // Get the position from arguments
        int position = getArguments() != null ? getArguments().getInt(ARG_POSITION) : 0;
    }
}

```

```

        // Set recipe details based on the position
        TextView textView = view.findViewById(R.id.recipe_detail);
        String[] details = {
            "Ingredients: Beef, Tomato Sauce, Pasta\nInstructions: Cook beef, add sauce, mix with pasta.",
            "Ingredients: Chicken, Cream, Parmesan\nInstructions: Cook chicken, add cream, mix with parmesan.",
            "Ingredients: Beef, Mushrooms, Sour Cream\nInstructions: Cook beef and mushrooms, add sour cream."
        };
        textView.setText(details[position]);

        return view;
    }
}

```

3. Create the Layout Files

res/layout/fragment_recipe_list.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <ListView
        android:id="@+id/recipe_list"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>
</LinearLayout>

```

res/layout/fragment_recipe_detail.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:id="@+id/recipe_detail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="16dp"/>
</LinearLayout>

```

4. Implement the Main Activity

MainActivity.java

```
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;
import androidx.fragment.app.FragmentManager;
import androidx.fragment.app.FragmentTransaction;

public class MainActivity extends AppCompatActivity implements
RecipeListFragment.OnRecipeSelectedListener {

    private boolean isTwoPane;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Check if the layout is using a two-pane layout
        isTwoPane = findViewById(R.id.fragment_detail_container) != null;

        // Add the RecipeListFragment
        RecipeListFragment recipeListFragment = new RecipeListFragment();
        FragmentManager fragmentManager = getSupportFragmentManager();
        FragmentTransaction fragmentTransaction =
fragmentManager.beginTransaction();
        fragmentTransaction.add(R.id.fragment_list_container, recipeListFragment);
        fragmentTransaction.commit();
    }

    @Override
    public void onRecipeSelected(int position) {
        if (isTwoPane) {
            // Update RecipeDetailFragment if in two-pane mode
            RecipeDetailFragment detailFragment =
RecipeDetailFragment.newInstance(position);
            FragmentManager fragmentManager = getSupportFragmentManager();
            FragmentTransaction fragmentTransaction =
fragmentManager.beginTransaction();
            fragmentTransaction.replace(R.id.fragment_detail_container, detailFragment);
            fragmentTransaction.commit();
        } else {
            // Start a new activity for the details if in single-pane mode
            // Intent to start RecipeDetailActivity
        }
    }
}
```

res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal">

    <!-- Container for RecipeListFragment -->
    <FrameLayout
        android:id="@+id/fragment_list_container"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1"/>

    <!-- Container for RecipeDetailFragment (only visible on tablets) -->
    <FrameLayout
        android:id="@+id/fragment_detail_container"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="2"/>
</LinearLayout>
```

Introduction to Fragment Manager

FragmentManager is a critical component in Android development that allows you to manage fragments within an activity. Fragments are modular sections of an activity's user interface, and the **FragmentManager** provides the tools needed to dynamically add, remove, or replace these fragments while the app is running.

Key Functions of FragmentManager

1. **beginTransaction()**

- **Purpose:** Initiates a new `FragmentTransaction` object. This is the starting point for performing operations on fragments.
- **Usage:** You need to begin a transaction before you can add, replace, or remove fragments.
- **Example:**

```
FragmentManager fragmentManager = getSupportFragmentManager();
FragmentTransaction transaction = fragmentManager.beginTransaction();
```

2. **add()**

- **Purpose:** Adds a fragment to a specified container view within the activity.
- **Usage:** Typically used to add a fragment to the activity layout for the first time or to add additional fragments.
- **Example:**

```
Fragment fragment = new RecipeListFragment();
```

```
transaction.add(R.id.fragment_container, fragment);
```

3. **replace()**

- **Purpose:** Replaces an existing fragment in a container with a new fragment.
- **Usage:** Useful for swapping fragments when the user interacts with the interface or when the application needs to display a different fragment.
- **Example:**

```
Fragment newFragment = new RecipeDetailFragment();  
transaction.replace(R.id.fragment_container, newFragment);
```

4. **remove()**

- **Purpose:** Removes a fragment from the activity's layout.
- **Usage:** Use this method to remove a fragment that is no longer needed, often in response to user actions or application logic.
- **Example:**

```
Fragment fragmentToRemove =  
getSupportFragmentManager().findFragmentById(R.id.fragment_container);  
transaction.remove(fragmentToRemove);
```

5. **commit()**

- **Purpose:** Finalizes and applies all the changes made during the transaction.
- **Usage:** Must be called to apply the changes made during the transaction. Without calling `commit()`, none of the fragment operations will be executed.
- **Example:**

```
transaction.commit();
```

Example

Consider an application with a single activity that dynamically displays different fragments based on user interaction. Here's how you might use `FragmentManager` to manage these fragments.

Use Case: In a recipe app, you want to switch between a list of recipes and detailed recipe information based on user selection.

1. Begin a Transaction

Start by obtaining the `FragmentManager` and beginning a new transaction.

```
FragmentManager fragmentManager = getSupportFragmentManager();  
FragmentTransaction transaction = fragmentManager.beginTransaction();
```

2. Add or Replace Fragments

Depending on user interaction, you might need to add or replace fragments. For instance, when a user selects a recipe from the list, replace the current fragment with the recipe detail fragment.

```
// Create a new fragment instance
Fragment recipeDetailFragment = RecipeDetailFragment.newInstance(selectedRecipeId);

// Replace the current fragment with the new one
transaction.replace(R.id.fragment_container, recipeDetailFragment);
```

3. Commit the Transaction

Finalize and apply the changes to make the fragment transaction take effect.

```
transaction.commit();
```

Difference between Activity and Fragments

Aspect	Activity	Fragment
Lifecycle	Independent, managed by the system.	Dependent on the host activity's lifecycle.
UI Flexibility	Entire screen UI.	Part of the UI, allowing modular and flexible UI.
Usage	Typically one activity per screen.	Multiple fragments can be used in one activity.
Reusability	Limited to the activity itself.	Can be reused across multiple activities.
Management	Managed by the system.	Managed by <code>FragmentManager</code> within an activity.

Menus in Android

Menus are essential for providing a consistent and familiar interface for users to interact with various features of an app. They help organize and present app actions in a way that is easy to access and use. In Android, menus come in three main types, each serving a different purpose:

Introduction

Menus in Android enhance user experience by offering a structured way to perform actions or navigate through different parts of the app. They provide a unified interface to manage app functionality.

Types of Menus in Android:

1. Options Menu:

- **Purpose:** Displays global actions that are relevant across different parts of the app. It is typically accessed via the menu button or an app bar.
- **Usage:** Used for actions that apply to the entire activity or app, like settings or saving data.
- **Implementation:**

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.options_menu, menu);
    return true;
}
```

2. Context Menu:

- **Purpose:** Appears when the user performs a long-press on a specific item or view, providing actions related to that particular item.
- **Usage:** Used for actions that are specific to the item under the user's long-press, like editing or deleting an item.
- **Implementation:**

```
@Override
public void onCreateContextMenu(ContextMenu menu, View v,
ContextMenuItem menuInfo) {
    super.onCreateContextMenu(menu, v, menuInfo);
    getMenuInflater().inflate(R.menu.context_menu, menu);
}
```

3. Popup Menu:

- **Purpose:** Provides a menu that is anchored to a specific view in the UI, showing actions related to that view.
- **Usage:** Ideal for displaying a small menu with actions that are specific to the element on which the menu is anchored, like item options in a list.
- **Implementation:**

```
PopupMenu popup = new PopupMenu(this, view);
popup.getMenuInflater().inflate(R.menu.popup_menu, popup.getMenu());
popup.show();
```

Implementing a Menu in an Application

To use menus in your app, follow these steps:

1. Define the Menu in XML:

- Create XML files for each menu type in the `res/menu` directory. Each file should define the menu items and their properties.

- Example (options_menu.xml):

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/action_settings"
        android:title="Settings"
        android:orderInCategory="100"
        android:showAsAction="never" />
    <item
        android:id="@+id/action_save"
        android:title="Save"
        android:orderInCategory="200"
        android:showAsAction="ifRoom" />
</menu>
```

2. Inflate the Menu:

- Use the `MenuInflater` to inflate the menu XML file in the corresponding activity or fragment.
- **Options Menu:** Override `onCreateOptionsMenu()` in your activity.
- **Context Menu:** Register the view for the context menu and override `onCreateContextMenu()`.
- **Popup Menu:** Create a `PopupMenu` object and use `getMenuInflater().inflate()` to load the menu XML.

Dialogs

Introduction

- Dialogs are small windows that prompt the user to make a decision or enter information. They are commonly used for alerts, confirmations, or input.

Creating a Dialog Fragment

- Extend `DialogFragment` to create a dialog that can be managed as part of an activity's fragment lifecycle.

```
public class MyDialogFragment extends DialogFragment {
    @NonNull
    @Override
    public Dialog onCreateDialog(Bundle savedInstanceState) {
        AlertDialog.Builder builder = new
        AlertDialog.Builder(getActivity());
        builder.setMessage("Are you sure?")
            .setPositiveButton("Yes", (dialog, id) -> {
                // Perform some action
            })
            .setNegativeButton("No", (dialog, id) -> {
                // Cancel the dialog
            });
        return builder.create();
    }
}
```



```
}
```

Setting a Dialog's Content

- Customize the dialog by setting a message, title, buttons, and custom layouts if needed.
- Example:

```
AlertDialog.Builder builder = new AlertDialog.Builder(getActivity());
builder.setTitle("Dialog Title")
    .setMessage("This is a dialog message.")
    .setPositiveButton("OK", (dialog, which) -> {
        // Action on OK
    })
    .setNegativeButton("Cancel", (dialog, which) -> {
        // Action on Cancel
    });
AlertDialog dialog = builder.create();
dialog.show();
```